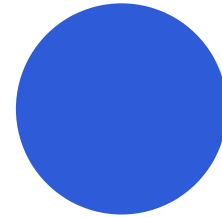
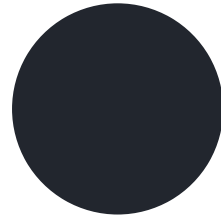
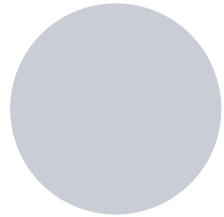


Build your own.

Day 3 — agents that keep going, and a server for a problem YOU care about.



Three questions before we build.

1

What makes a good tool docstring?

It tells the LLM WHEN to use the tool — not just what it does.

2

Who executes the tool call — the LLM or the client?

The client. Always. The LLM only decides.

3

What is Stop & Parse, and why does client.py do it?

Stop the LLM before it invents a result · run the real tool · resume with truth.

Today's promise: by the end of this session you have a running MCP server for a problem you care about.

You already understand every server on this list.

Real production MCP servers — same decorator, same docstrings, different domain.

GitHub MCP

read repos, open issues, create PRs — from a chat

Postgres MCP

ask your database questions in plain English

Brave Search MCP

give any LLM live web search

Filesystem MCP

your Day 1 `read_file` — production-grade

Slack MCP

send messages, search conversations

The blue one is personal. You built its core tool two days ago. ~20,000 servers are indexed on public registries — all this pattern. Browse: mcpservers.org

What is an agent?

The three levels — you've already built two of them.

1 — Chatbot reads prompt, returns text · *ChatGPT answering a question*

2 — Tool use calls ONE tool, returns answer · *your client.py from Day 2*

3 — Agent calls tools IN SEQUENCE, stops when done · *what we build today*

The difference is who controls the loop. Day 2: YOU wrote the loop — one call, stop. An agent: the LLM decides how many calls, and when it's done.

Enough theory.

Let's build an agent.

Five lines replace yesterday's fifty — then you build your own.

Demo, stage 1: an agent with no tools.

Yesterday you wrote the loop in 50 lines. A framework does it in 5. First: empty-handed.

```
from smolagents import ToolCallingAgent, LiteLLMModel

model= LiteLLMModel(model_id="ollama_chat/llama3.2:3b")
agent= ToolCallingAgent(tools= [], model=model)

agent.run("What is France known for, what's the weather in its
          capital, and what does notes.txt say?")
```

France facts

OK — training data

Weather in Paris

HALLUCINATED

notes.txt

cannot read files

Stage 2: two lines connect your Day 1 server.

server.py is already running in Terminal 1 — the server YOU built.

```
from smolagents import MCPClient

_mcp = MCPClient({"url": "http://127.0.0.1:8000/mcp/"})
server_tools = [*_mcp.__enter__()]

tools = server_tools      # was: tools = []
```

Same question, run again: weather is now REAL (wttr.in, live) · notes.txt is now REAL (your file). The agent discovered read_file and get_weather exactly like client.py did — the framework just hides the plumbing.

Stage 3: one more tool — Wikipedia.

A local @tool this time — note the docstring style: it tells the LLM WHEN.

```
from smolagents import tool

@tool
def wikipedia_summary(topic: str) -> str:
    """Get a short summary of any topic from Wikipedia.
    Use this for facts, history, definitions, or general knowledge.
    Args:
        topic: The topic to look up, e.g. 'France', 'Tel Aviv'.
    """
    ... # free REST call — en.wikipedia.org/api/rest_v1

tools = [*server_tools, wikipedia_summary]
```


Now watch it think.

One question, three tools, zero manual chaining.

“What is France known for, what's the weather in its capital, and what does notes.txt say?”

Step 1

```
wikipedia_summary('France')
```

...learns the capital is Paris

Step 2

```
get_weather('Paris')
```

uses 'Paris' from step 1 — nobody told it to

Step 3

```
read_file('notes.txt')
```

real contents · then: final answer, no 4th call

Point at the token counts: 1,245 → 2,569. The context grows every step — everything the agent learned is visible to it in the next Thought. That's the TAO loop, running itself.

Now I build a new server. Watch my decisions.

New domain, from scaffold.py — the code is easy; the design choices are the lesson.

Docstring FIRST

If I can't write a clear docstring, my tool design is wrong.

One thing per tool

List AND read? No — two tools. The LLM chains them.

Test after every tool

Connect client.py, verify, only then add the next.

Free APIs only

wtr.in, Open-Meteo, Wikipedia, exchangerate — no keys in class.

Goal: 2–3 working tools in 20 minutes. Then it's your turn.

DO NOW — build YOUR server (30 min)

Pick something you'd actually use. Start from scaffold.py.

1

Pick a domain — todo list · currency · Wikipedia · movies · your own — ideas.md has 8 starters.

2

One tool first — docstring first · get it WORKING before adding a second.

3

Test with client.py — or yesterday's agent — ask a question only YOUR tool can answer.

4

Finished early? — Connect your server to Goose — a real desktop client, 5-minute config.

tool not showing → restart server · LLM ignores your tool → improve the docstring (WHEN, not what) · API needs a key → pick a free one

Show us. (5 minutes each)

3–4 volunteers. The demo format:



One sentence

what your server does



One question

run it through client.py or Goose



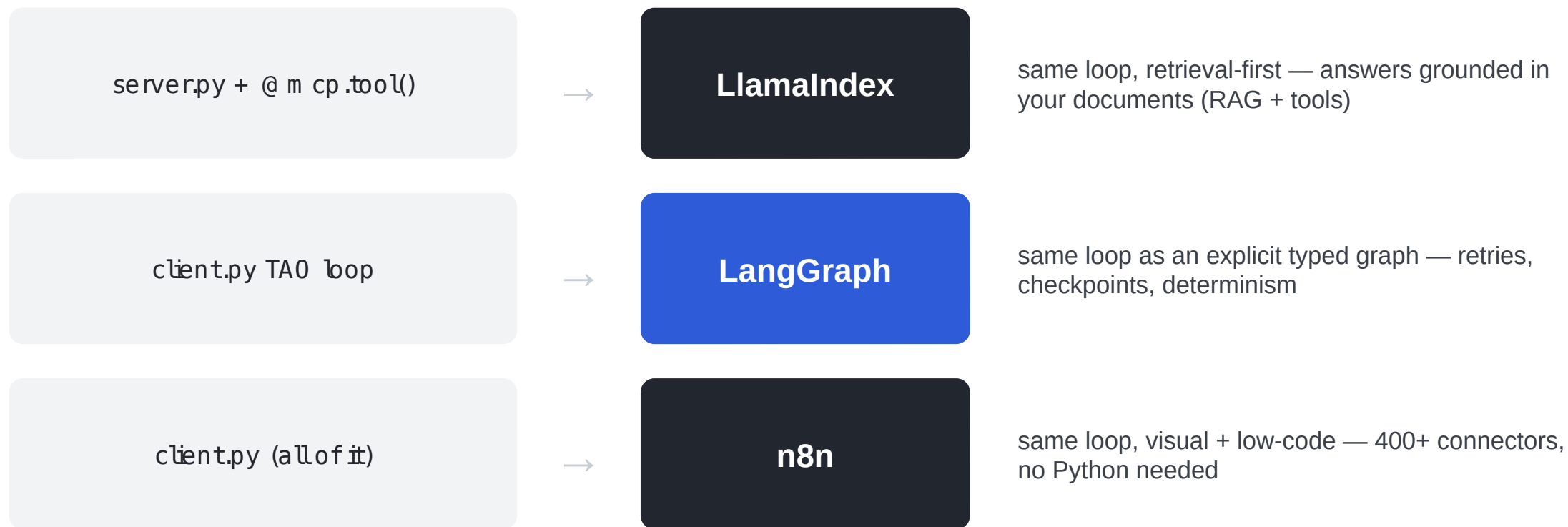
One terminal

show the tool actually being called

Nervous? The bar is: it runs once. That's a demo.

What you built → what professionals reach for.

All of them are YOUR loop, packaged differently. The architecture is identical.



Also: smolagents (today's demo) · AutoGen (multi-agent). Same loop, every time.

Your toolbox from here.

Goose

free desktop chat, connects to any MCP server — block.github.io/goose

Claude Desktop

Anthropic's app — connects via STDIO

MCP Inspector

browser debugger: `npx @modelcontextprotocol/inspector python your_server.py`

mcpservers.org

~20,000 servers indexed — all the pattern you now know

July 28 spec

blog.modelcontextprotocol.io — stateless core, Tasks, MCP Apps. Your server keeps working.

You know the whole machine now.

a server with tools

a client that discovers them

an LLM that decides

a loop that executes

Every AI assistant you use — ChatGPT with plugins, Claude with tools, Gemini with extensions — is running this same loop. You understand it from the inside.

The only thing between you and building something real:

What problem do you want to solve?